

# Theoretical Foundations

## A Defense Study Companion

Hybrid Post-Quantum Key Exchange for WireGuard (X25519 + ML-KEM-768)

Mihai-Cristian Farcaș | Supervisor: Prof. Dr. Darabant Sergiu Adrian

**How to use this.** This is a briefing book, not a second thesis. Each entry is short on purpose: enough to answer a committee question with confidence and to follow up one level deeper if pressed. Read it alongside the thesis — every entry names the section(s) where the concept actually does work in your argument. The grey “*In the thesis*” line is your jump label; the shaded “**If they probe**” boxes pre-empt the obvious follow-up.

**The one-paragraph anchor.** Everything below serves a single claim: *WireGuard’s key exchange relies on X25519, an elliptic-curve Diffie–Hellman that a large quantum computer would break via Shor’s algorithm; this thesis adds ML-KEM-768 (FIPS 203) alongside X25519 in a hybrid “concatenate-then-KDF” construction inside BoringTun, behind a `pq` compile flag, so that a session stays confidential as long as either primitive holds — and quantifies the cost on real hardware.* Keep that sentence in your head; each topic is a spoke off it.

Build: `tectonic foundations.tex`. Theme matches the slide deck and speaker script.

---

## Contents

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Networking and VPN Foundations</b>                           | <b>2</b> |
| 1.1      | The OSI model and layer 3 . . . . .                             | 2        |
| 1.2      | WireGuard . . . . .                                             | 2        |
| 1.3      | IPsec . . . . .                                                 | 3        |
| 1.4      | OpenVPN . . . . .                                               | 3        |
| 1.5      | TLS . . . . .                                                   | 4        |
| 1.6      | MTU: the IPv4/IPv6 maximum transmission unit . . . . .          | 4        |
| 1.7      | PMTUD (Path MTU Discovery) . . . . .                            | 4        |
| <b>2</b> | <b>Classical Cryptographic Primitives</b>                       | <b>5</b> |
| 2.1      | Diffie–Hellman key exchange . . . . .                           | 5        |
| 2.2      | The integer factorization problem . . . . .                     | 5        |
| 2.3      | RSA . . . . .                                                   | 5        |
| 2.4      | The elliptic-curve discrete logarithm problem (ECDLP) . . . . . | 6        |
| 2.5      | Curve25519, X25519, and the 32-byte key . . . . .               | 6        |
| 2.6      | ChaCha20-Poly1305 . . . . .                                     | 6        |
| 2.7      | AES-GCM . . . . .                                               | 7        |
| 2.8      | BLAKE2s . . . . .                                               | 7        |
| 2.9      | HMAC and HKDF key derivation . . . . .                          | 7        |
| <b>3</b> | <b>The Noise Protocol Framework and the WireGuard Handshake</b> | <b>8</b> |
| 3.1      | The Noise Protocol Framework . . . . .                          | 8        |
| 3.2      | The Noise IK pattern (and WireGuard’s IKpsk2) . . . . .         | 8        |
| 3.3      | Unknown-Key-Share (UKS) attacks . . . . .                       | 8        |
| <b>4</b> | <b>The Quantum Threat</b>                                       | <b>9</b> |
| 4.1      | Shor’s algorithm . . . . .                                      | 9        |
| 4.2      | What Shor breaks — and what it does not (Grover) . . . . .      | 9        |
| 4.3      | Harvest-now, decrypt-later (HNDL) . . . . .                     | 10       |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>5</b> | <b>Post-Quantum Cryptography: Standards and Constructions</b> | <b>10</b> |
| 5.1      | NIST . . . . .                                                | 10        |
| 5.2      | The NIST PQC Standardization Project . . . . .                | 10        |
| 5.3      | The Module Learning With Errors (MLWE) problem . . . . .      | 11        |
| 5.4      | CRYSTALS-Kyber, ML-KEM, and Kyber-768 . . . . .               | 11        |
| 5.5      | FIPS 203 . . . . .                                            | 12        |
| 5.6      | The Number-Theoretic Transform (NTT) . . . . .                | 12        |
| 5.7      | IND-CPA and IND-CCA2 security . . . . .                       | 12        |
| 5.8      | The Fujisaki–Okamoto (FO) transformation . . . . .            | 13        |
| 5.9      | The road not taken: Classic McEliece and Saber . . . . .      | 13        |
| <b>6</b> | <b>Assurance: Verification and Testing</b>                    | <b>13</b> |
| 6.1      | EasyCrypt and formal verification . . . . .                   | 13        |
| 6.2      | NIST Known Answer Tests (KATs) . . . . .                      | 14        |
| 6.3      | Historical security exploits in cryptographic code . . . . .  | 14        |
| <b>7</b> | <b>Implementation Concepts</b>                                | <b>14</b> |
| 7.1      | C FFI (Foreign Function Interface) . . . . .                  | 14        |
| 7.2      | Rust <code>no_std</code> . . . . .                            | 15        |
| <b>8</b> | <b>Evaluation Methodology</b>                                 | <b>15</b> |
| 8.1      | Criterion benchmarking . . . . .                              | 15        |
| 8.2      | Confidence intervals . . . . .                                | 15        |

---

## 1 Networking and VPN Foundations

This first group locates your work on the map: *where* in the stack WireGuard sits, what it competes with, and the packet-size constraints that post-quantum messages bump into.

### 1.1 The OSI model and layer 3

*In the thesis:* §2.2 (WireGuard’s design); motivates why the work is transparent to applications.

*In one sentence:* WireGuard operates at the network layer (layer 3), so it secures whole IP packets and everything above them, invisibly to applications.

The OSI reference model is a seven-layer abstraction of networking: physical (1), data link (2), network (3), transport (4), session (5), presentation (6), and application (7). The two reference points you need are **layer 2**, which moves frames between directly-connected nodes (Ethernet, MAC addresses), and **layer 3**, the *network* layer, which routes *packets* across networks using IP addresses.

WireGuard is a **layer-3** tunnel: it presents a virtual network interface, takes whole IP packets, encrypts each one, and ships it inside a UDP datagram to the peer. Because it works on IP packets, any transport- or application-layer protocol (TCP, QUIC, HTTP, DNS) rides over it without modification. This is the contrast with OpenVPN’s TAP mode, which can tunnel at layer 2 (Ethernet frames). The practical consequence for your thesis: your change touches only how the *tunnel key* is established; the data-plane that carries layer-3 traffic is untouched.

### 1.2 WireGuard

*In the thesis:* §2.2 and §2.2.1 (protocol + PSK slot); the whole thesis modifies it.

*In one sentence:* A deliberately minimal layer-3 VPN: one round-trip handshake built on the Noise framework, a fixed modern cipher suite, and roughly 4,000 lines of code.

WireGuard (Jason Donenfeld, 2017; in the mainline Linux kernel since 5.6) is a VPN built around radical simplicity. Its design choices, each of which you should be able to name:

- **Fixed cryptography, no negotiation.** Unlike TLS or IPsec there is no cipher-suite negotiation — and therefore no downgrade attacks. The suite is Curve25519 (X25519) for key agreement, ChaCha20-Poly1305 for authenticated encryption, BLAKE2s for hashing and keyed MAC, and an HMAC-based KDF. This rigidity is exactly why adding post-quantum security means *editing the protocol*, not flipping a negotiated option.
- **1-RTT Noise IK handshake.** One message each way establishes a session; see §3 below.
- **Cryptokey routing.** Each peer is bound to a set of “allowed IPs”; the public key *is* the identity, which doubles as the routing decision.
- **Stateless under load / DoS resistance.** A cookie-reply mechanism (the `mac1/mac2` fields) lets a loaded responder defer state allocation until a client proves it can receive.
- **Silence is a virtue.** It runs over UDP and does not reply to unauthenticated packets.

BoringTun is Cloudflare’s *userspace* Rust implementation of this protocol — the codebase you extended.

**If they probe.** “Why is no negotiation a security feature?” Negotiation is where downgrade attacks live (e.g. TLS’s history). WireGuard’s fixed suite removes that class entirely; the trade-off is that algorithm changes require a versioned protocol change — which is precisely the surface your **pq** flag and new message types 5/6 add.

### 1.3 IPsec

*In the thesis:* Background contrast to WireGuard; RFC 8784 parallels your PSK-slot baseline.

*In one sentence:* The older, standardized, kernel-level layer-3 VPN suite — powerful but complex, with a separate key-exchange protocol (IKEv2).

IPsec is a suite (not one protocol) that secures IP at layer 3 using two data formats — ESP (Encapsulating Security Payload, confidentiality + integrity) and AH (Authentication Header, integrity only) — keyed by the **IKEv2** handshake. It is mature and ubiquitous but notoriously complex: many modes, options, and negotiation steps. Two points matter for you. First, it is the canonical “heavyweight” contrast that motivated WireGuard’s minimalism. Second, the IETF’s interim post-quantum answer for IPsec, **RFC 8784**, mixes a *pre-shared post-quantum key* into IKEv2 — which is conceptually identical to your **PSK-slot baseline**: get PQ confidentiality without touching the asymmetric handshake, at the cost of out-of-band key distribution.

### 1.4 OpenVPN

*In the thesis:* Background contrast; representative TLS-based userspace VPN.

*In one sentence:* A long-established userspace VPN that rides on TLS/OpenSSL and can tunnel at layer 2 or 3 — flexible and battle-tested, but heavier and slower than WireGuard.

OpenVPN builds its tunnel on top of a TLS session (historically via OpenSSL), typically over UDP. Its flexibility (certificates, pluggable ciphers, TAP layer-2 or TUN layer-3 modes) is also its weight: a larger codebase, more configuration, and more overhead per packet than WireGuard. In your framing it is the “general-purpose, TLS-based” point of comparison, against which WireGuard’s fixed, lean Noise design stands out — and it inherits TLS’s post-quantum story rather than defining its own.

## 1.5 TLS

*In the thesis: §3.5 (related work: PQ via TLS PSK delivery); industry precedent for hybrids.*

*In one sentence:* The transport-layer security protocol of the web; TLS 1.3 already ships a deployed hybrid X25519+ML-KEM-768 key exchange, which is the strongest real-world precedent for your design.

TLS secures byte streams *above* the transport layer (it sits on TCP, or on QUIC at the application layer). TLS 1.3 reduced the handshake to one round trip and pruned legacy options. The reason it belongs in your defense is the precedent: in 2024 Chrome and Cloudflare turned on a **hybrid** key exchange named **X25519MLKEM768** — the exact same primitives you combine, combined the same way (concatenate the two shared secrets, feed them through the KDF). So your thesis is not exotic; it transplants an approach the wider industry has already validated at scale into a protocol (WireGuard) that did not yet have it. Related-work §3.5 also discusses delivering a PQ secret to WireGuard *through* a TLS channel.

**If they probe.** “Why not just rely on TLS being post-quantum?” TLS protects different traffic (application byte streams), not the WireGuard tunnel itself. A WireGuard session’s confidentiality depends on *WireGuard’s* handshake; fixing it requires changing that handshake, which is the contribution.

## 1.6 MTU: the IPv4/IPv6 maximum transmission unit

*In the thesis: §5.5 (message size analysis) and §9.2 (deployment).*

*In one sentence:* The largest packet a link will carry in one piece; post-quantum handshake messages are bigger, so you have to show they still fit.

The MTU is the largest payload a link-layer technology will carry in a single frame — classically 1500 bytes on Ethernet. Exceed it and the packet must be fragmented or dropped. The two IP versions differ crucially:

- **IPv4** routers may fragment in transit; the guaranteed minimum reassembly is small (576 bytes is the conventional safe figure).
- **IPv6** forbids in-network fragmentation — only the sender may fragment — and guarantees a **minimum MTU of 1280 bytes** on every link.

Your post-quantum messages grow because ML-KEM-768 adds an encapsulation key ( $\approx 1184$  B) and a ciphertext ( $\approx 1088$  B). The key result you defend: the PQ initiation ( $\approx 1332$  B) and response ( $\approx 1180$  B) each still fit inside a single 1500-byte Ethernet datagram, and even within the 1280-byte IPv6 floor the response fits and the initiation needs only minimal fragmentation. Fragmentation only becomes a concern below a path MTU of roughly 1360 bytes.

## 1.7 PMTUD (Path MTU Discovery)

*In the thesis: §9.2 (deployment considerations and failure modes).*

*In one sentence:* The mechanism hosts use to learn the smallest MTU along a path; it fails silently (“blackholes”) when the ICMP signals it relies on are filtered.

PMTUD lets a sender discover the smallest MTU on the path to a destination. In IPv4 the sender sets the “Don’t Fragment” bit; a router that needs to fragment instead drops the packet and returns an ICMP “fragmentation needed” message carrying the next-hop MTU. IPv6 does the analogous thing with ICMPv6 “Packet Too Big.” The well-known failure mode — a **PMTUD blackhole** — happens when firewalls filter those ICMP messages: the oversized packets just vanish with

no feedback, and the connection hangs. This matters to you because if a path MTU were small enough to fragment the PQ handshake, and ICMP were filtered, the handshake could stall. Your defense is that the messages are sized to avoid fragmentation on normal paths in the first place, so PMTUD pathologies do not bite. Note also: lowering the WireGuard interface MTU does *not* help, because the large packets are the UDP-encapsulated handshake on the outer path, not inner tunnel traffic.

## 2 Classical Cryptographic Primitives

These are the pieces WireGuard already uses, plus the hard problems they rest on. Know which ones quantum computers threaten (the public-key ones) and which they do not (the symmetric ones) — that split is the entire rationale for the thesis.

### 2.1 Diffie–Hellman key exchange

*In the thesis: §2.1 (the quantum threat to public-key crypto); foundation for X25519.*

*In one sentence:* The 1976 method by which two parties derive a shared secret over a public channel; WireGuard’s whole handshake is a sequence of these.

Diffie–Hellman (DH) lets two parties who have never met agree on a shared secret visible to no eavesdropper. In the classic finite-field form, public parameters are a prime  $p$  and generator  $g$ ; Alice sends  $g^a \bmod p$ , Bob sends  $g^b \bmod p$ , and both compute  $g^{ab}$ . An eavesdropper sees  $g^a$  and  $g^b$  but recovering  $g^{ab}$  requires solving the **discrete logarithm problem** (DLP). DH provides *key agreement*, not authentication — on its own it is vulnerable to a meddler-in-the-middle, which is why WireGuard authenticates by mixing long-term static keys into the handshake. Crucially, DH (in both finite-field and elliptic-curve form) is broken by Shor’s algorithm; this is the quantum problem your thesis addresses.

### 2.2 The integer factorization problem

*In the thesis: §2.1 (why classical public-key crypto falls to quantum).*

*In one sentence:* Given a large  $N = p \cdot q$ , recover the primes  $p$  and  $q$  — believed hard classically, easy for a quantum computer.

Factoring underpins RSA. The best classical algorithm, the General Number Field Sieve, runs in *sub-exponential* time — slow enough that 2048-bit moduli are safe against classical attackers. The catch is “classical.” Shor’s algorithm factors in polynomial time on a quantum computer, collapsing the security margin. You cite factorization as one of the two classical assumptions (with the discrete log) that quantum computing dissolves.

### 2.3 RSA

*In the thesis: §2.1 (representative classical public-key scheme under quantum threat).*

*In one sentence:* The 1977 public-key cryptosystem (encryption and signatures) whose security reduces to integer factorization — and which Shor’s algorithm breaks.

RSA (Rivest–Shamir–Adleman) was the first practical public-key system. A public key is  $(N, e)$  with  $N = pq$ ; the private key is the matching exponent  $d$ . Security rests on factoring being hard: knowing  $N$  should not reveal  $p$  and  $q$ . WireGuard does *not* use RSA — it uses elliptic-curve crypto — but RSA is the textbook example of a factoring-based scheme and the headline victim of Shor in resource estimates (e.g. Gidney–Ekerå, 2019, estimate RSA-2048 in hours on  $\sim 20$  million noisy qubits). It frames the threat your work responds to.

## 2.4 The elliptic-curve discrete logarithm problem (ECDLP)

*In the thesis: §2.1 and §2.2 (the hardness behind X25519).*

*In one sentence:* Given points  $P$  and  $Q = kP$  on an elliptic curve, recover the scalar  $k$  — the hard problem that makes X25519’s small keys secure (classically).

On an elliptic curve, the group operation is point addition; “exponentiation” becomes scalar multiplication  $kP$ . The ECDLP is recovering  $k$  from  $P$  and  $Q = kP$ . Its key property: the best classical attacks (Pollard’s rho) cost about  $\sqrt{n}$  for group order  $n$ , i.e. fully exponential, with *no* sub-exponential shortcut like the number field sieve. That is why elliptic-curve keys can be far smaller than RSA keys at equal classical security — 256-bit curves versus 3072-bit RSA. But ECDLP, like ordinary DLP and factoring, also falls to Shor’s algorithm; in fact a 256-bit ECDLP needs *fewer* logical qubits than RSA-2048, so elliptic-curve crypto is arguably the more urgent quantum target.

## 2.5 Curve25519, X25519, and the 32-byte key

*In the thesis: §2.2 (WireGuard primitives), §5.3 (where ML-KEM joins it in the hybrid).*

*In one sentence:* Curve25519 is the elliptic curve; X25519 is the Diffie–Hellman function over it; the public key and shared secret are each 32 bytes — this is the classical half of your hybrid.

**Curve25519** (Bernstein, 2006) is a Montgomery-form elliptic curve over the prime field  $\mathbb{F}_p$  with  $p = 2^{255} - 19$ , designed for speed and for resistance to implementation pitfalls (it is “safe” by construction — no invalid-curve or twist problems if used correctly, and amenable to constant-time code). **X25519** is the Diffie–Hellman function built on it (RFC 7748): each side has a 32-byte private scalar (with specific bits “clamped”) and a 32-byte public key (the curve point’s  $u$ -coordinate); the shared secret is the 32-byte  $u$ -coordinate of the scalar product. Those ubiquitous **32-byte** quantities — WireGuard public keys, X25519 outputs — are 256-bit values. In your hybrid, the X25519 shared secret is one of the two inputs concatenated before the KDF; ML-KEM-768’s 32-byte shared secret is the other. X25519 is the component a quantum attacker would eventually defeat via Shor; ML-KEM is what keeps the session safe if that happens.

**If they probe.** “Why keep X25519 at all if it’s quantum-broken?” Two reasons: (1) ML-KEM is young and lattice assumptions are less battle-tested than ECDLP, so a hybrid stays secure if *either* holds; (2) it preserves classical security and WireGuard’s existing authentication unchanged. Hybrid = belt and braces, which is exactly NIST’s and industry’s recommended migration posture.

## 2.6 ChaCha20-Poly1305

*In the thesis: §2.2 (transport encryption), §8.3 (throughput is unaffected by your change).*

*In one sentence:* WireGuard’s authenticated-encryption cipher for the data channel: a software-fast stream cipher (ChaCha20) plus a one-time MAC (Poly1305); your work does not touch it.

ChaCha20-Poly1305 (Bernstein; RFC 8439) is an **AEAD** (Authenticated Encryption with Associated Data) construction: ChaCha20 is a 256-bit-key stream cipher and Poly1305 is a fast one-time message authenticator; together they give confidentiality *and* integrity in one pass. WireGuard chose it over AES-GCM because it is fast and *constant-time in pure software*, with no dependence on AES hardware acceleration — which matters on devices like the Raspberry Pi. The defense-relevant point: this is symmetric crypto, so it is *not* broken by Shor (only mildly weakened by Grover), and your thesis leaves the entire transport path on ChaCha20-Poly1305 untouched. That is why your measured throughput is statistically unchanged — only the one-time handshake cost grows.



## 2.7 AES-GCM

*In the thesis: Contrast to ChaCha20-Poly1305 (§2.2); explains a WireGuard design choice.*

*In one sentence:* The other mainstream AEAD: AES in counter mode plus the GHASH authenticator — excellent with hardware AES, which is why WireGuard chose ChaCha20-Poly1305 instead for hardware-agnostic speed.

AES-GCM (NIST SP 800-38D) combines AES in counter mode with the GHASH universal hash for authentication. It is the dominant AEAD in TLS and is extremely fast *where AES-NI hardware instructions exist*. Its weaknesses in software are the flip side: without hardware acceleration, constant-time AES is harder and slower, and GCM is unforgiving of nonce reuse. This contrast explains WireGuard’s preference for ChaCha20-Poly1305 and is worth knowing because your Raspberry Pi results lean on the “no AES hardware needed” argument. Like all symmetric ciphers it is quantum-resistant with adequate key length.

## 2.8 BLAKE2s

*In the thesis: §2.2 (hashing/MAC), §5.4 (the KDF that mixes your hybrid secret).*

*In one sentence:* WireGuard’s hash function — fast, modern, and used for hashing, the keyed MAC, and the KDF; it is what stirs your concatenated secret into the chain.

BLAKE2s is a cryptographic hash (a member of the BLAKE2 family derived from the SHA-3 finalist BLAKE) optimized for 8–32-bit platforms, producing up to a 256-bit digest. WireGuard uses it pervasively: as the transcript hash that binds the handshake, as a *keyed* MAC (the `mac1/mac2` cookie fields), and as the compression function inside its HMAC-based KDF. For your thesis it is the hash underneath the key derivation in §5.4: when you concatenate the X25519 and ML-KEM secrets, it is a BLAKE2s-based HMAC chain that absorbs them into the chaining key. Being a hash, it is quantum-resistant (Grover only halves preimage security, which 256-bit output absorbs).

## 2.9 HMAC and HKDF key derivation

*In the thesis: §2.5 (concatenate-then-KDF) and §5.4 (key derivation details).*

*In one sentence:* HMAC is a hash-based message authentication code; HKDF is the “extract-then-expand” key-derivation function built from it — the machinery that turns your two raw shared secrets into uniform session keys.

**HMAC** (RFC 2104) builds a MAC from any hash  $H$  via  $\text{HMAC}(k, m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$ . **HKDF** (Krawczyk; RFC 5869) is a KDF with two stages: *extract* condenses raw, possibly-non-uniform input keying material into a uniform pseudorandom key, then *expand* stretches that into as many output keys as needed. WireGuard uses an HKDF-style HMAC-BLAKE2s chain throughout its handshake.

This is the heart of why “concatenate-then-KDF” is sound. You set the KDF input keying material to the concatenation  $ss_{\text{X25519}} \parallel ss_{\text{ML-KEM}}$ . A standard KDF security property (and the NIST SP 800-56C Rev. 2 guidance for combining secrets) guarantees the output is pseudorandom as long as *at least one* input secret is unknown to the attacker. That single property is the formal backbone of your hybrid’s “secure if either primitive holds” claim — no new cryptography invented, just a correct application of an established combiner.

**If they probe.** “Why concatenate rather than XOR or nest the secrets?” Concatenation feeding a KDF is the construction NIST SP 800-56C analyses and the TLS hybrid uses; it provably preserves security if either secret is strong. XOR can be fragile if one secret is adversarially controlled; concatenate-then-KDF is the conservative, standardized choice.

## 3 The Noise Protocol Framework and the WireGuard Handshake

### 3.1 The Noise Protocol Framework

*In the thesis: §2.3 (Noise) and §5.3 (where you insert ML-KEM into the pattern).*

*In one sentence:* A toolkit (Trevor Perrin) for building handshake protocols out of a fixed alphabet of Diffie–Hellman and mixing operations; WireGuard is one specific Noise pattern.

Noise is not a protocol but a *framework* for constructing two-party handshakes. A handshake is written as a short *pattern* of tokens — e.g. **e** (send an ephemeral key), **s** (send a static key), **ee**, **es**, **se**, **ss** (perform the corresponding Diffie–Hellman) — and the framework prescribes how each operation mixes into two running values: a *chaining key* (feeds the KDF) and a *handshake hash* (binds the transcript for authentication). The elegance is that security properties (forward secrecy, identity hiding, authentication) follow from the pattern. The catch for post-quantum work: Noise’s vocabulary assumes *Diffie–Hellman*, and a KEM is not a DH (see §4.1). So inserting ML-KEM means stepping slightly outside vanilla Noise — exactly the careful integration your implementation performs and documents.

### 3.2 The Noise IK pattern (and WireGuard’s IKpsk2)

*In the thesis: §2.3, §5.3 (hybrid handshake), §5.2 / §2.2.1 (the PSK slot = the “psk2”).*

*In one sentence:* IK is the specific Noise handshake WireGuard uses: the initiator knows the responder’s static key in advance (K) and transmits its own static key, encrypted, during the handshake (I).

In Noise’s two-letter naming, the first letter describes the *initiator’s* static key and the second the *responder’s*:

- **I** — the initiator’s static key is transmitted (*immediately*, encrypted) to the responder during the handshake.
- **K** — the responder’s static key is *known* to the initiator beforehand (out of band; in WireGuard, from the peer’s configured public key).

This fits a VPN perfectly: a client already has the server’s public key in its config (K), and proves its own identity in the first message (I). WireGuard extends IK with an optional pre-shared key mixed at the second position, making it effectively **IKpsk2**; that “psk2” slot is precisely the **PSK-slot baseline** you compare against (Option 2). Your **hybrid** (Option 1) instead threads ML-KEM into the existing two-message flow: the initiator carries the ML-KEM encapsulation key, the responder returns the ciphertext, and the resulting KEM shared secret is mixed into the same chaining key alongside the X25519 DH results — no third message added.

**If they probe.** “Did you add a round trip?” No. ML-KEM’s encapsulation key rides in the existing initiation and its ciphertext rides in the existing response, so the handshake stays 1-RTT. Only the message *sizes* and the per-handshake *compute* grow; the message *count* does not.

### 3.3 Unknown-Key-Share (UKS) attacks

*In the thesis: §7.4 (authentication and remaining limitations).*

*In one sentence:* An attack where two honest parties end up sharing a key but disagree about *who* they are talking to; prevented by binding both identities into the transcript — which WireGuard already does.



In a UKS attack the adversary does not learn the key, but it tricks Alice into believing she shares a key with the attacker while she actually shares it with Bob (or vice versa) — a confusion of identities rather than of key material. The standard defense is to fold both parties’ identities (their static public keys) into the authenticated handshake transcript, so the derived key is cryptographically committed to “who talked to whom.” WireGuard does this: static public keys are mixed into the handshake hash. You raise UKS in the security analysis to show you understand the authentication model and that your change preserves it — your modification adds confidentiality material (the KEM secret) without weakening the identity binding. It also sets up the honest limitation: *authentication* still rests on X25519 static keys, so it is not itself post-quantum (addressed next, and in §7.4/§9.3).

## 4 The Quantum Threat

### 4.1 Shor’s algorithm

*In the thesis: §2.1 (the quantum threat) — the reason the thesis exists.*

*In one sentence:* The 1994 quantum algorithm that solves factoring and discrete logs (including elliptic-curve) in polynomial time, breaking RSA, DH, and X25519.

Peter Shor’s algorithm (1994) is the single result that endangers today’s public-key crypto. Its engine is quantum *period-finding*: factoring  $N$  reduces to finding the period of the function  $x \mapsto a^x \bmod N$ , and the **Quantum Fourier Transform** extracts that period efficiently — turning a sub-exponential classical problem into a polynomial-time quantum one. The same period-finding machinery solves the discrete logarithm in any finite abelian group, which sweeps in finite-field DH *and* the elliptic-curve discrete log behind X25519. So a sufficiently large, fault-tolerant quantum computer would break WireGuard’s key exchange outright.

The essential caveat: “sufficiently large and fault-tolerant.” Current devices are noisy and far too small; published estimates put RSA-2048 at millions of physical qubits with error correction. No such machine exists today — which is exactly why the threat model is not “tomorrow” but *harvest-now, decrypt-later* (§4.3).

**If they probe.** “If no quantum computer can do this yet, why migrate now?” Because encrypted traffic captured today can be stored and decrypted later once the machine exists. Data with multi-year confidentiality needs is already at risk; the migration must precede the computer, not follow it. NIST/NSA timelines target the early 2030s.

### 4.2 What Shor breaks — and what it does not (Grover)

*In the thesis: §2.1; justifies why only the handshake, not the transport, needs PQ.*

*In one sentence:* Shor demolishes public-key (asymmetric) crypto; symmetric crypto and hashes are only mildly dented by Grover’s algorithm, so doubling key sizes suffices there.

This split is the load-bearing argument for your whole scope:

- **Asymmetric / public-key** (RSA, DH, ECDH/X25519, ECDSA) — *broken* by Shor. These need replacing. WireGuard’s key exchange lives here.
- **Symmetric & hashes** (ChaCha20, AES, BLAKE2s, SHA-2) — only *weakened* by Grover’s algorithm, which gives at most a quadratic speedup for brute-force search. Effective security halves: a 256-bit cipher retains  $\sim 128$ -bit post-quantum security, which is ample.

Therefore the only quantum-vulnerable part of WireGuard is the **handshake** (the X25519 key

agreement); the ChaCha20-Poly1305 transport is fine as-is. This is precisely why your contribution targets the handshake *only* and leaves the data plane untouched — a scoping decision you can now justify from first principles.

### 4.3 Harvest-now, decrypt-later (HNDL)

*In the thesis:* §1 (introduction / motivation) and §2.1.

*In one sentence:* The real, present-day threat: an adversary records encrypted traffic now and decrypts it once a quantum computer arrives — so long-lived secrets need protection before the machine exists.

HNDL (also “store-now-decrypt-later”) reframes the quantum threat from a future event into a present obligation. An adversary with storage and patience captures today’s encrypted sessions and simply waits; when a cryptographically-relevant quantum computer arrives, the recorded X25519 handshakes are broken and the session keys — hence all that captured traffic — are recovered. Anything that must stay confidential for years (state secrets, health and financial records, long-term credentials) is therefore *already* exposed. This is the urgency argument and it is specifically a *confidentiality* threat, which is why a hybrid that protects the key exchange is valuable even though it leaves classical *authentication* in place: forging authentication requires a quantum computer at handshake time and cannot be done retroactively against recorded traffic.

## 5 Post-Quantum Cryptography: Standards and Constructions

The core of your contribution’s “new half.” Be fluent here above all.

### 5.1 NIST

*In the thesis:* §2.4 (ML-KEM is a NIST/FIPS standard).

*In one sentence:* The U.S. National Institute of Standards and Technology — the body that ran the post-quantum competition and publishes the FIPS standards your work implements.

NIST is a U.S. federal agency that, among much else, standardizes cryptography through its Federal Information Processing Standards (FIPS) publications — AES (FIPS 197) and SHA-3 (FIPS 202) came through NIST processes, and so did ML-KEM (FIPS 203). Its imprimatur is why ML-KEM, rather than raw Kyber, is the thing to deploy: it is the vetted, frozen, interoperable specification. When the committee hears “FIPS 203,” they hear “the standard,” and your choice to target it rather than a research draft signals engineering maturity.

### 5.2 The NIST PQC Standardization Project

*In the thesis:* §2.4 and §3 (related work situates competing schemes).

*In one sentence:* The multi-year open competition (2016–) that evaluated post-quantum algorithms and selected the first standards in 2022, published as FIPS in 2024.

NIST launched the Post-Quantum Cryptography Standardization Project in 2016, soliciting public submissions for quantum-resistant key-establishment and signature schemes. The timeline you should be able to sketch:

- **2017** — 69 candidates accepted into Round 1.
- **2019 / 2020** — narrowed to Round 2 (26) then Round 3 (7 finalists + 8 alternates).

- **July 2022** — first selections: **CRYSTALS-Kyber** for key encapsulation, plus CRYSTALS-Dilithium, Falcon, and SPHINCS+ for signatures.
- **August 2024** — standards published: **FIPS 203** (ML-KEM, from Kyber), FIPS 204 (ML-DSA, from Dilithium), FIPS 205 (SLH-DSA, from SPHINCS+).
- A separate “Round 4” continued evaluating additional KEMs for diversity (BIKE, Classic McEliece, HQC; SIKE was eliminated after being broken), with HQC selected in 2025.

The process matters rhetorically: ML-KEM is not a lab curiosity but the survivor of years of public cryptanalysis — which is the assurance your hybrid leans on for its post-quantum half.

### 5.3 The Module Learning With Errors (MLWE) problem

*In the thesis: §2.4 (the hardness assumption under ML-KEM).*

*In one sentence:* The lattice problem ML-KEM’s security rests on: distinguishing noisy linear equations over a module from random — believed hard for classical *and* quantum computers.

Learning With Errors (LWE) asks you to recover a secret  $\mathbf{s}$  from many samples  $(\mathbf{a}_i, \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i)$  where each  $e_i$  is small random “noise” — equivalently, to distinguish those noisy inner products from uniform randomness. The noise is what makes Gaussian elimination fail and the problem hard; its hardness connects to worst-case lattice problems. Plain LWE has large keys, so practical schemes use structure: **Ring-LWE** works over a polynomial ring (compact but more structure assumed), and **Module-LWE** interpolates between the two by working over a rank- $k$  module of that ring. Kyber/ML-KEM uses MLWE over  $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$  with module rank  $k$  tied to the security level. The selling point: MLWE lets you tune security by changing  $k$  while reusing the same ring arithmetic, and no efficient quantum algorithm is known against it — unlike the abelian-group problems Shor demolishes.

### 5.4 CRYSTALS-Kyber, ML-KEM, and Kyber-768

*In the thesis: §2.4 and §5.3 (the post-quantum half of the hybrid).*

*In one sentence:* Kyber is the MLWE-based KEM that won the NIST competition; ML-KEM is its standardized form (FIPS 203); ML-KEM-768 / “Kyber-768” is the security level you use.

**CRYSTALS-Kyber** (Cryptographic Suite for Algebraic Lattices) is a key-encapsulation mechanism built on MLWE. **ML-KEM** is its NIST-blessed final form; the names are often used interchangeably, but “ML-KEM” denotes the exact FIPS-203 algorithm (with a few tweaks from the competition Kyber). As a KEM it exposes three operations:

- **KeyGen**  $\rightarrow$  (encapsulation key / public, decapsulation key / secret);
- **Encaps**(encapsulation key)  $\rightarrow$  (ciphertext, shared secret) — run by the party that has the peer’s public key;
- **Decaps**(decapsulation key, ciphertext)  $\rightarrow$  shared secret.

**ML-KEM-768** (“Kyber-768”) is the middle parameter set: module rank  $k = 3$ , targeting NIST security **category 3** (roughly AES-192). Its sizes — which you should have at your fingertips because they drive the MTU analysis — are an encapsulation key of **1184 bytes**, a ciphertext of **1088 bytes**, and a **32-byte** shared secret. You chose the 768 level as the balanced, widely-recommended default (it is the level in TLS’s X25519MLKEM768), strong enough for category 3 without the larger messages of ML-KEM-1024.

**If they probe.** “Why ML-KEM-768 and not 512 or 1024?” 768 is NIST category 3 and the de-facto industry default (it is the level chosen for the deployed TLS hybrid). 512 trades security margin for size; 1024 buys category 5 at noticeably larger messages. 768 is the balanced choice and keeps your handshake within one Ethernet datagram.

## 5.5 FIPS 203

*In the thesis:* §2.4 (the standard your implementation targets).

*In one sentence:* The August 2024 NIST standard that defines ML-KEM precisely — the specification your RustCrypto `ml-kem` dependency implements.

FIPS 203, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” finalized on 13 August 2024, is the authoritative definition of ML-KEM: the algorithms, the three parameter sets (512/768/1024), the exact byte encodings, and the internal hashing. Targeting a published FIPS — rather than a moving research target — is what makes your implementation interoperable and your choice defensible. The crate you depend on (`ml-kem`, RustCrypto) tracks this standard, which is why correctness can be checked against NIST’s official test vectors (§6 below).

## 5.6 The Number-Theoretic Transform (NTT)

*In the thesis:* §2.4; explains why ML-KEM is fast and underlies your latency numbers.

*In one sentence:* An FFT over a finite field that turns slow polynomial multiplication into cheap coefficient-wise products — the reason Kyber is fast, and the reason its parameters  $q = 3329$ ,  $n = 256$  look the way they do.

Kyber’s arithmetic lives in  $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ , and multiplying two degree-255 polynomials naively is  $O(n^2)$ . The NTT is the finite-field analogue of the FFT: it evaluates a polynomial at the powers of a root of unity so that multiplication becomes *pointwise*, giving  $O(n \log n)$ . Kyber’s parameters are chosen *precisely* to make this work:  $q = 3329$  is prime with  $q \equiv 1 \pmod{256}$ , so a length-256 transform exists. (A subtlety worth knowing: because  $q \not\equiv 1 \pmod{512}$ ,  $X^{256} + 1$  does not split into linear factors, so Kyber uses an “incomplete” NTT down to 128 quadratic factors with base-case products — but the speed win stands.) Public keys even travel in NTT domain. This is the mechanistic reason your ML-KEM operations are fast (tens of microseconds on the M4); your benchmark chapter measures the practical payoff.

## 5.7 IND-CPA and IND-CCA2 security

*In the thesis:* §2.4 (KEM security) and §7.2 (hybrid security argument).

*In one sentence:* The two standard “indistinguishability” security definitions; ML-KEM must meet the stronger one (IND-CCA2), because a real attacker can submit chosen ciphertexts.

These are game-based security *definitions*, framed as an attacker trying to distinguish encryptions:

- **IND-CPA** (chosen-plaintext): the attacker may encrypt plaintexts of its choosing but cannot ask for decryptions. The minimal sensible bar.
- **IND-CCA2** (adaptive chosen-ciphertext): the attacker additionally has a *decryption oracle* and may use it adaptively (except on the challenge). This models the real world, where an adversary can send crafted ciphertexts to a peer and observe behavior.

A KEM used in a live protocol must be **IND-CCA2**, because a network attacker can inject ciphertexts. ML-KEM’s underlying public-key scheme (K-PKE) is only IND-CPA; the upgrade to IND-CCA2 is done by the Fujisaki-Okamoto transform (next). In your security chapter, the

KEM’s IND-CCA2 guarantee is one of the two pillars; the KDF combiner then carries that strength into the hybrid session key.

## 5.8 The Fujisaki–Okamoto (FO) transformation

*In the thesis: §2.4 (how ML-KEM achieves CCA security).*

*In one sentence:* The generic recipe that upgrades a merely IND-CPA encryption scheme into an IND-CCA2 KEM by re-encrypting on decapsulation and rejecting anything that doesn’t match — this is how ML-KEM gets its strong security.

The FO transform turns a weak (IND-CPA) public-key scheme into a strong (IND-CCA2) KEM. The idea: the shared secret is derived deterministically from the message, so on **Decaps** the receiver *re-encrypts* the recovered message and checks it reproduces the received ciphertext exactly. A mismatch means tampering, and the KEM refuses to output the real key. ML-KEM uses the modern variant with **implicit rejection**: instead of returning a visible error (which can leak information), it returns a *pseudorandom* key derived from a secret seed, so a chosen-ciphertext attacker learns nothing from malformed queries. This re-encryption check is also why decapsulation costs a bit more than encapsulation — relevant when you read your per-operation benchmarks.

**If they probe.** “Where does CCA security come from in ML-KEM?” From the FO transform applied to the IND-CPA scheme K-PKE, with implicit rejection. The decapsulator re-encrypts and compares; on failure it returns a pseudorandom key, leaking nothing.

## 5.9 The road not taken: Classic McEliece and Saber

*In the thesis: §2.4 / §3 (context for why ML-KEM, not an alternative).*

*In one sentence:* Two other NIST KEM candidates — code-based McEliece (ultra-conservative but enormous keys) and lattice-based Saber (close runner-up to Kyber) — useful as foils for “why ML-KEM.”

**Classic McEliece** is a *code-based* KEM (security from error-correcting codes, an assumption studied since 1978 and untouched by quantum attacks). Its virtue is conservatism; its problem is a public key of hundreds of kilobytes to a megabyte — impossible to carry in a 1500-byte WireGuard handshake. **Saber** is a lattice KEM very close in spirit to Kyber but based on Module Learning-With-Rounding with power-of-two moduli (so it avoids the NTT); it was a Round-3 finalist that NIST did not select. Both are worth a sentence because they show you understand the design space: McEliece’s giant keys make it a non-starter for an MTU-bound handshake, and Saber lost to Kyber on NIST’s balance of security, performance, and analysis — so ML-KEM-768 is the natural, standards-aligned, size-feasible choice for WireGuard.

## 6 Assurance: Verification and Testing

How anyone trusts that the cryptography is correct — the standard’s, the library’s, and yours.

### 6.1 EasyCrypt and formal verification

*In the thesis: §2.4 / §7 (why ML-KEM as a primitive can be trusted).*

*In one sentence:* A proof assistant for machine-checking cryptographic security proofs; ML-KEM has been formally verified with it, which is part of why the primitive is trustworthy and not your burden to re-prove.

EasyCrypt is an interactive theorem prover specialized for the *game-based* proofs cryptographers write by hand — it mechanically checks the sequence of “game hops” that bounds an attacker’s advantage, removing the human errors that have historically crept into paper proofs. It matters to your defense as a boundary marker: the security of ML-KEM *as a primitive* has been the subject of formal-verification efforts (e.g. machine-checked Kyber/ML-KEM implementations in the Jasmin/Formosa-crypto line of work using EasyCrypt), so you inherit a strongly-vetted building block. Your contribution is the *correct integration* of that block into WireGuard, not a re-proof of the lattice mathematics — a clean scoping statement if asked “did you prove ML-KEM secure?”

## 6.2 NIST Known Answer Tests (KATs)

*In the thesis: §6.5 (test harness) — how you verify your integration is correct.*

*In one sentence:* Deterministic test vectors — fixed seed in, exact expected bytes out — that confirm a crypto implementation matches the standard.

A Known Answer Test pins down a deterministic computation: given a fixed seed (so the otherwise-random key generation and encapsulation become reproducible), the implementation must produce exactly the published keys, ciphertext, and shared secret. NIST ships KAT files with FIPS 203 for precisely this. Their value is that they catch byte-level mistakes — wrong encoding, endianness, parameter mix-ups — that ordinary “it ran” testing misses. In your test harness, KAT agreement (via the `ml-kem` crate’s conformance) gives confidence that the ML-KEM operations you wired into the handshake are the *real* ML-KEM, and your own integration tests then confirm two patched peers actually complete a hybrid handshake and derive matching keys.

## 6.3 Historical security exploits in cryptographic code

*In the thesis: §6 (rationale for a Rust implementation and a minimal, gated diff).*

*In one sentence:* The catalogue of real-world cryptographic failures — memory bugs and logic slips — that motivates implementing in memory-safe Rust and keeping the change small and optional.

A short rogues’ gallery you can cite to justify engineering choices:

- **Heartbleed** (OpenSSL, 2014) — a missing bounds check let attackers read adjacent memory, leaking private keys. A *memory-safety* failure in C.
- **Apple “goto fail”** (2014) — a duplicated `goto` skipped TLS signature verification entirely. A *logic* failure.
- **Debian OpenSSL PRNG** (2008) — a well-meaning edit gutted entropy, making keys guessable. A *review/process* failure.

The lessons map directly onto your decisions: BoringTun is **Rust**, whose memory safety eliminates the Heartbleed class by construction; you keep the post-quantum change **small, auditable, and behind a pq feature flag** so the default build is byte-for-byte the upstream code (no risk to existing users); and you lean on **standardized primitives with KATs and formal verification** rather than hand-rolling crypto, which is where the logic failures live. This is your answer to “why should we trust your code?”

# 7 Implementation Concepts

## 7.1 C FFI (Foreign Function Interface)

*In the thesis: §6.1–§6.2 (BoringTun architecture and technology stack).*



*In one sentence:* The mechanism by which Rust exposes a C-callable API; BoringTun ships one, and your `pq` flag is designed to leave that ABI untouched in the default build.

An FFI lets code in one language call code compiled from another; the C ABI is the lingua franca. Rust exposes C-callable functions with `extern "C"` and `#[no_mangle]`, and such boundaries are **unsafe** because the compiler cannot verify the foreign side's assumptions (lifetimes, null pointers, thread safety). BoringTun is consumed as a library through exactly such a C FFI (it is embedded into other applications, not only run as a daemon). Why this matters to you: keeping the post-quantum work behind a compile-time feature means the *default* build presents the same FFI surface and the same wire format as upstream — existing integrators are unaffected, and only an explicitly PQ-enabled build changes behavior. That is a deployment-safety argument the committee will appreciate.

## 7.2 Rust `no_std`

*In the thesis:* §6.2 (technology stack); relevant to constrained-device deployability.

*In one sentence:* A Rust mode that drops the OS-dependent standard library and uses only the core language, enabling crypto to run on embedded targets — which is why a KEM library can reach routers and small devices.

By default Rust programs link `std`, which assumes an operating system (heap allocator, files, threads). A crate marked `#![no_std]` uses only `core` (the allocation-free language essentials), optionally adding `alloc` for heap types — letting it run on microcontrollers and inside kernels. RustCrypto primitives, including the `ml-kem` family, are written to be `no_std`-capable. The relevance to your thesis is forward-looking: although BoringTun itself targets hosted platforms (it uses `std`), the fact that the post-quantum primitive is `no_std`-friendly means the approach is portable down to constrained devices — consistent with your Raspberry Pi evaluation and with WireGuard's ambition to run everywhere.

# 8 Evaluation Methodology

How your performance claims are made trustworthy rather than anecdotal.

## 8.1 Criterion benchmarking

*In the thesis:* §6.5 (test/bench harness) and all of §8 (results).

*In one sentence:* Rust's statistical benchmarking library — it warms up, samples many iterations, detects outliers, and reports a mean with a confidence interval rather than a single noisy number.

Criterion is the de-facto statistics-driven benchmarking framework for Rust. Its methodology is the point: it runs a *warm-up* phase (to settle caches and CPU frequency), then collects many timed samples, fits and reports a *mean estimate with a confidence interval*, detects *outliers*, and can compare against a saved baseline to flag regressions. This is what lifts your numbers from “I ran it once and got 247  $\mu$ s” to a defensible statistical statement. You extended BoringTun's existing Criterion-based tests to measure the handshake under the three configurations (vanilla, PSK-slot, hybrid) and the individual ML-KEM operations, on both the Apple M4 and the Raspberry Pi 5.

## 8.2 Confidence intervals

*In the thesis:* §8.1 (latency results are reported as mean  $\pm$  95% CI).

*In one sentence:* A range that quantifies measurement uncertainty: a 95% CI means the procedure captures the true mean 95% of the time — so non-overlapping intervals signal a real difference,

not noise.

A confidence interval expresses how sure you are about an estimate. A **95% CI** for the mean is constructed so that, across repeated experiments, 95% of such intervals would contain the true mean. Criterion derives these by *bootstrap resampling* the collected timings (no assumption of a normal distribution). For your defense the practical reading is comparative: when the hybrid's latency interval sits clearly above vanilla's and the two do not overlap, the ~64% increase on the M4 (and ~25% on the Pi) is a *statistically real* effect, not measurement jitter — whereas the near-identical vanilla and PSK-slot intervals overlap, correctly showing the PSK approach adds no asymmetric-handshake cost. Reporting the interval, not just the mean, is what makes the comparison honest.

**If they probe.** “Are your performance differences significant?” Yes — they are reported as means with 95% bootstrap confidence intervals from Criterion over many samples; the hybrid's interval is disjoint from vanilla's, so the overhead is a real effect, while vanilla vs. PSK-slot overlap, correctly indicating no handshake-asymmetric cost.

---

**Closing orientation.** If you remember nothing else: *Shor breaks the public-key handshake but not the symmetric transport; a KEM is not a Diffie–Hellman*, so ML-KEM is woven into Noise IK as encapsulation-key-out / ciphertext-back rather than a DH exchange; *the two secrets are concatenated and run through the BLAKE2s KDF*, which is secure if *either* holds; and *the cost is a bigger, slower handshake but an unchanged data plane*, measured honestly with Criterion confidence intervals. Everything in this companion is a more detailed footnote to those four facts.